Business Requirements Document (BRD)

Project 4: GenAI Payment Failure Intelligence & Resolution (RAG)

1. Executive Summary

Payment systems process millions of transactions daily across channels such as UPI, cards, and net banking. A significant percentage of these transactions fail due to various reasons including bank declines, network issues, insufficient funds, and fraud checks.

Failure reasons are often stored as:

- error codes
- gateway messages
- bank response descriptions
- logs

These are **technical and difficult to interpret**.

Operations and support teams struggle to:

- understand failure reasons
- identify root causes
- suggest next actions
- respond to customers quickly

The proposed solution is to build a **GenAI-powered Payment Failure Intelligence Copilot** using **RAG (Retrieval Augmented Generation)**.

The system will:

- ingest payment logs, error codes, and documentation
- retrieve relevant failure context
- generate human-readable explanations
- recommend resolution steps

2. Business Problem

Payment failures impact:

- customer experience
- merchant revenue
- platform reliability

Challenges include:

- failure reasons are cryptic (e.g., "Do Not Honor", "U16", "RTO")
- multiple sources (gateway, bank, network)
- lack of centralized knowledge
- manual debugging required

This results in:

1. Delayed issue resolution
2. Increased support workload
3. Poor customer satisfaction

3. Business Objectives

The objective is to build a **GenAI assistant for payment failure analysis**.

Key objectives:

1. Interpret payment failure codes and logs.
2. Provide human-readable explanations.
3. Suggest actionable next steps.
4. Reduce support resolution time.
5. Improve payment success rate insights.

4. Project Scope

In Scope

- Ingest payment failure logs and documentation
- Build vector database of failure knowledge
- Implement RAG pipeline
- Enable natural language queries
- Generate failure explanations and recommendations

Out of Scope

- Real-time payment processing integration
- Automated retry systems
- Live bank integrations

5. Stakeholders

Stakeholder	Role
Payment Operations Team	Analyze failures
Customer Support Team	Respond to users
Engineering Team	Debug payment issues
Product Team	Improve success rates

6. User Personas

Payment Support Agent

Responsibilities:

- handle failed transaction queries
- respond to customers

Pain Points:

- difficult error codes
- lack of clear explanation

Expected Benefit:

- AI-generated explanations + solutions

Payment Analyst

Responsibilities:

- analyze failure trends
- identify root causes

Expected Benefit:

- faster failure analysis

8. Functional Requirements

FR1 – Data Ingestion

The system must ingest:

- payment failure logs
- gateway error codes
- bank response messages
- troubleshooting documents

Example:

Error Code: U16
Message: Transaction declined by bank
Gateway: Razorpay

FR2 – Document Processing

The system must:

- clean and structure logs
- convert text into chunks
- prepare data for embeddings

FR3 – Embedding Generation

The system must convert failure data into embeddings using:

- Sentence Transformers / OpenAI embeddings

FR4 – Vector Database

The system must store embeddings in:

- FAISS / ChromaDB

FR5 – Query Interface

Users must ask questions such as:

Why did transaction fail with code U16?

FR6 – Context Retrieval (RAG)

The system must:

- retrieve relevant failure logs and documentation
- pass context to LLM

FR7 – GenAI Response Generation

The system must generate:

- explanation of failure
- root cause
- suggested action

Example output:

Failure Reason:
The transaction was declined by the issuing bank.

Possible Causes:
• Insufficient funds
• Bank security restrictions

Suggested Actions:
• Ask customer to retry
• Use a different payment method

FR8 – Multi-Source Explanation

The system must combine:

- bank response
- gateway logs
- internal documentation

FR9 – Dashboard (Optional)

Display:

- top failure reasons
- failure categories
- suggested fixes

8. Non-Functional Requirements

Requirement	Description
Performance	<5 sec response time
Scalability	Handle 50k logs locally
Usability	Simple UI
Local Execution	Must run locally

9. Data Requirements

Data Sources

- payment logs (CSV)
- error code documentation
- gateway failure descriptions

Sample Dataset

TransactionID	ErrorCode	Message	Gateway
TXN001	U16	Declined by bank	Razorpay
TXN002	5	Do not honor	Visa

Dataset Size

10,000 – 50,000 records

10. System Architecture

Failure Logs + Docs
↓
Text Processing
↓
Embeddings
↓
Vector DB (FAISS)
↓
User Query
↓
Retriever
↓
LLM
↓
Explanation + Resolution

11. Technology Stack

Component	Technology
Programming	Python
RAG Framework	LangChain / LlamaIndex
Embeddings	Sentence Transformers
Vector DB	FAISS / ChromaDB
LLM	OpenAI / Ollama
UI	Streamlit

12. Success Metrics

Requirement	Description
Performance	<5 sec response time
Scalability	Handle 50k logs locally
Usability	Simple UI
Local Execution	Must run locally

13. Risks and Mitigation

Risk	Mitigation
Incorrect explanations	Provide source references
Ambiguous error codes	Use multi-source context
LLM hallucination	Strict prompt design

14. Deliverables

Employees must deliver:

1. Data ingestion pipeline
2. Vector database
3. RAG pipeline
4. GenAI explanation engine
5. Query interface
6. Documentation

15. Timeline

Phase	Duration
Analysis	2 days
Design	2 days
Development	1 days
Testing	1 days
Demo	1 day

Total: **7 days**